FICHE TUTEUR · PROJET SUIVI D'INCIDENTS BACKBONE

Semaine 8

Tests, finitions et soutenance

Durée : 5 jours Tests automatisés Qualité du code Documentation Présentation

Intentions pédagogiques

Fin de parcours. Trois objectifs : **fiabiliser** (tests), **transmettre** (README, code lisible) et **convaincre** (démonstration). La soutenance est aussi l'occasion d'évaluer la progression sur l'ensemble du stage, pas seulement la semaine.

Demandez-lui de **casser volontairement** une règle métier pour voir un test échouer : c'est ce qui fait comprendre l'utilité des tests.

Déroulé conseillé

Jour	Activité
J1	Premiers tests : invités, connexion, pages accessibles.
J2	Tests métier : création, résolution, suivi, autorisations, filtres, rapport.
J3	Pint, relecture du code, correction des 5 irritants.
J4	README testé par une autre personne. Répétition de la démonstration.
J5	**Soutenance** (15 min de démo + 15 min de questions). Bilan du stage.

Corrigé : fichiers de référence

- `tests/Feature/IncidentTest.php` (10 tests, 43 assertions)
- `database/factories/*` (états `admin()`, `resolu()`)
- `README.md`

Exemple de test d'autorisation :

```
public function test_un_technicien_ne_gere_pas_les_utilisateurs_ni_ne_supprime(): void
{
    $incident = Incident::factory()->create();

    $this->actingAs(User::factory()->create());

    $this->get(route('users.index'))->assertForbidden();
    $this->delete(route('incidents.destroy', $incident))->assertForbidden();
}
```

Piège rencontré dans le corrigé : un test du rapport hebdomadaire vérifiait qu'un incident d'une autre semaine n'apparaissait pas, mais cet incident était **encore ouvert** et apparaissait donc légitimement dans « encore ouverts ». Le test était faux, pas l'application. Bon exemple à raconter : **un test qui échoue n'a pas toujours raison**.

Erreurs fréquentes

Erreur	Symptôme	Piste
`RefreshDatabase` oublié	Tests qui dépendent de l'ordre d'exécution	Base vierge à chaque test
Tests sur la base MySQL de dev	Données de démo effacées	Vérifier `phpunit.xml` (SQLite en mémoire)
SQL spécifique MySQL	Tests qui plantent sous SQLite	Calcul en PHP ou SQL portable
Dates aléatoires des factories	Tests instables (« flaky »)	Fixer `debut_at` dans le test
README non testé	Étape manquante (`key:generate` , création de base)	Le faire suivre par quelqu'un d'autre
Démo improvisée	Bugs en direct	Scénario écrit et répété deux fois

Questions de soutenance (avec réponses attendues)

1. **Pourquoi des enums plutôt que des chaînes pour statut et sévérité ?** *Attendu :* valeurs contrôlées, libellés et couleurs centralisés, erreurs détectées par l'éditeur.
2. **Que se passe-t-il si deux techniciens résolvent le même incident en même temps ?** *Attendu :* le dernier enregistrement gagne (« last write wins »). Piste : verrouillage optimiste, champ `version` . Il doit savoir **identifier** la limite.
3. **Ton application est-elle protégée contre l'injection SQL ? Comment ?** *Attendu :* Eloquent utilise des requêtes préparées ; attention à `selectRaw` / `whereRaw` avec des données utilisateur.
4. **Comment ferais-tu pour recevoir automatiquement les alarmes de la supervision ?** *Attendu :* une route API (token), validation, création d'incident ; éviter les doublons pour une même alarme.
5. **Si la table `incidents` atteint 5 millions de lignes, qu'est-ce qui ralentit en premier ?** *Attendu :* la recherche `LIKE '%…%'` , le tableau de bord (agrégations) ; pistes : index, index FULLTEXT, mise en cache des indicateurs.
6. **Qu'as-tu appris de plus important pendant ce stage ?** *Attendu :* réponse personnelle ; valoriser le recul (méthode, sécurité, SQL…), pas la liste des fonctionnalités.

Grille d'évaluation de la semaine (sur 20)

Critère	Points
Au moins 10 tests pertinents, tous verts	5
Tests lisibles (Préparer / Agir / Vérifier, factories avec états)	2
Code formaté, sans code mort ni `dd()`	2
README complet et vérifié par un tiers	3
Démonstration fluide, scénario réaliste	4
Réponses aux questions, recul sur les limites	4

Bilan du stage (grille globale)

Compétence	Acquis	En cours	Non acquis
Modéliser une base de données relationnelle	☐	☐	☐
Écrire des requêtes SQL (JOIN, GROUP BY, agrégats)	☐	☐	☐
Lire un plan d'exécution (`EXPLAIN`), choisir un index	☐	☐	☐
Migrations, modèles et relations Eloquent	☐	☐	☐
Routes, contrôleurs, vues Blade	☐	☐	☐
Validation et règles métier	☐	☐	☐
Sécurité web (CSRF, XSS, mass assignment, autorisations)	☐	☐	☐
Tests automatisés	☐	☐	☐
Git et travail incrémental	☐	☐	☐
Communication : expliquer et défendre ses choix	☐	☐	☐

Pour la suite

Les pistes d'évolution de la fiche apprenant (pièces jointes, disponibilité des liaisons, API de supervision…) peuvent servir de sujet de prolongation de stage ou de projet pour un nouvel apprenant qui reprendrait le code existant, ce qui est un excellent exercice de lecture de code.